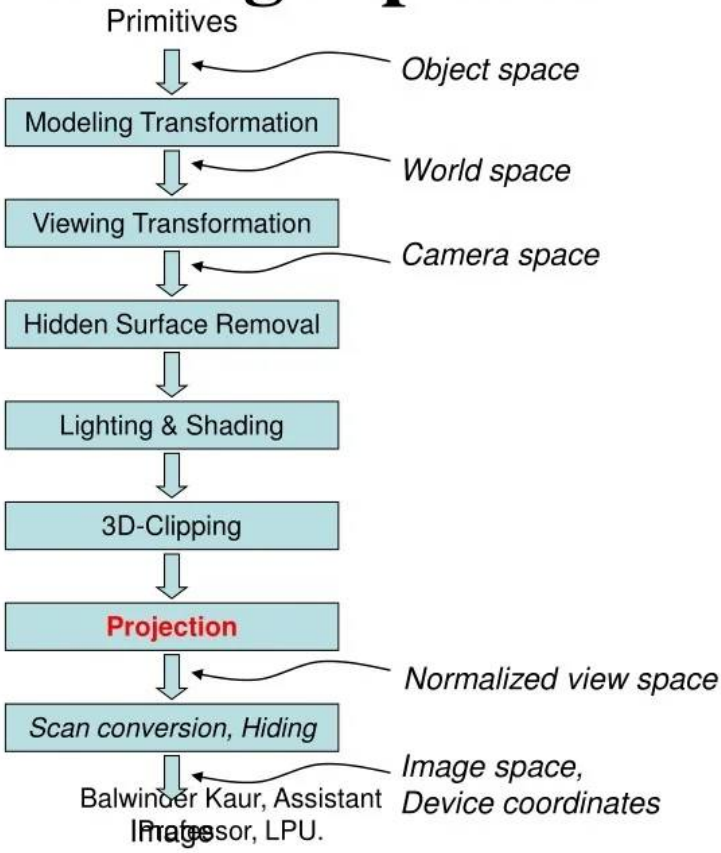
3D Viewing Pipeline

3D Viewing Pipeline



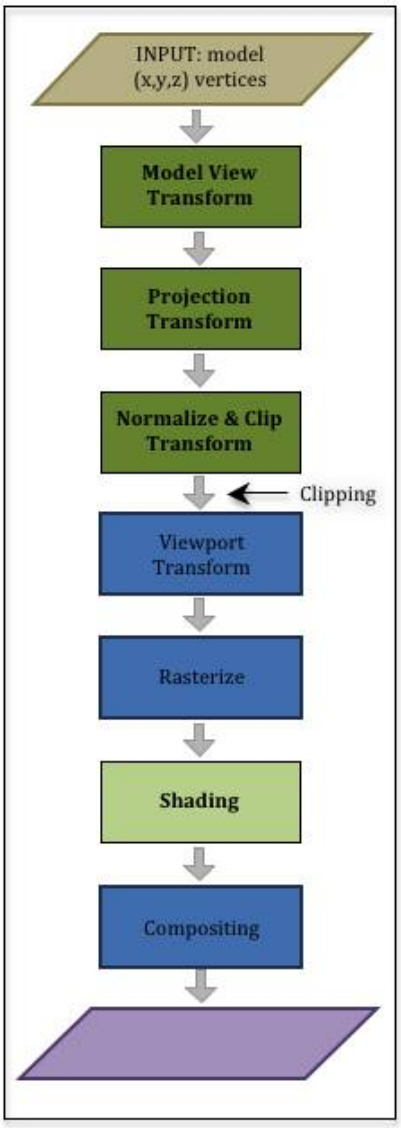
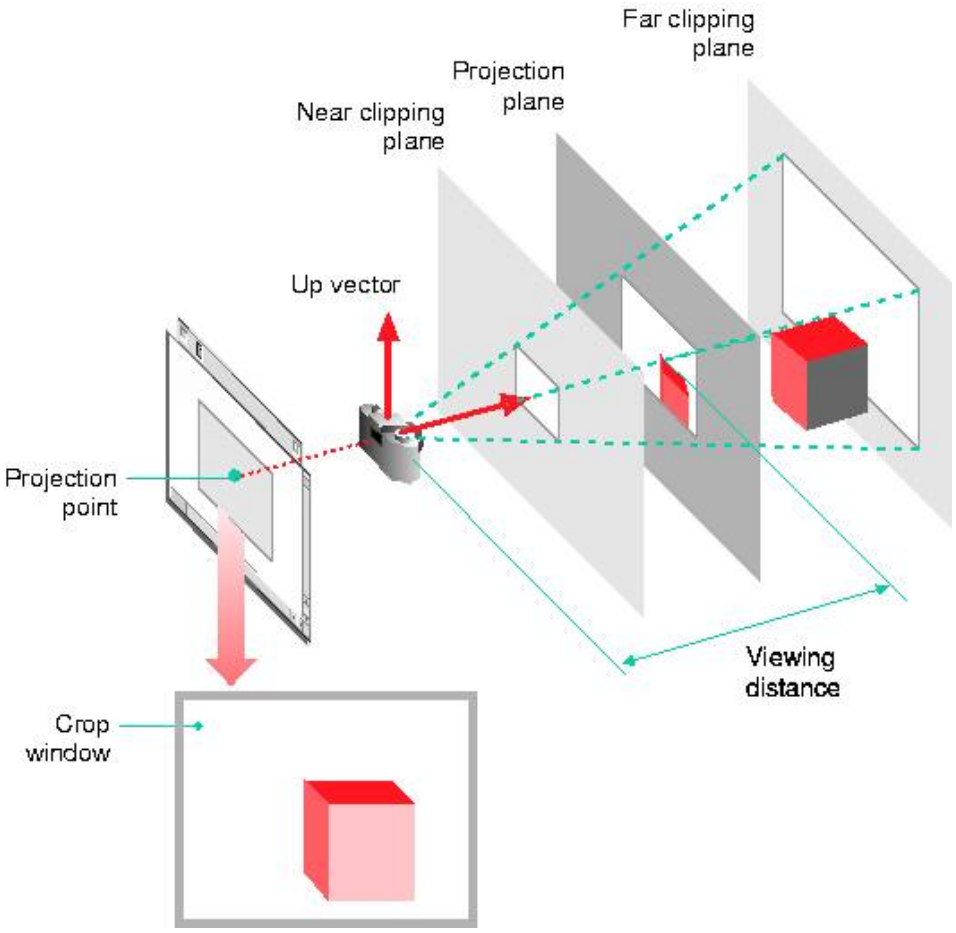

INPUT: model
(x,y,z) vertices
Model View
Transform
Projection
Transform
Normalize & Clip
Transform
Clipping
Viewport
Transform
Rasterize
Shading
Compositing

figure 216
Perspective camera



Below is a **complete, structured, and point-by-point explanation** of **3D Viewing** and all the concepts shown in your slides, **without skipping any item**.

---

1. What is 3D Viewing?

3D viewing is the process of converting a mathematical description of a three-dimensional scene into a two-dimensional image that can be displayed on a screen.
This process involves **geometric transformations, visibility determination, projection, and rendering**.

A 3D scene is not drawn directly. Instead, it passes through a **viewing pipeline**, where coordinates are transformed step by step from object space to screen space.

---

2. Major Procedures in 3D Viewing

2.1 Generating a 3D Scene

- Objects are defined using **geometric primitives** such as:
  - Points
  - Lines
  - Polygons (most commonly triangles)
- Complex objects are represented as a **set of surfaces forming a closed boundary**.

- This boundary encloses the interior of the object.

2.2 Generating the Interior of 3D Objects (If Required)

- Needed in applications like:
  - Medical imaging
  - Solid modeling
  - Volume rendering
- Interior data can be represented using:
  - Voxels
  - Constructive Solid Geometry (CSG)
  - Implicit surfaces

2.3 Projecting Object Surfaces onto the Display Device

- This is done using the **3D viewing pipeline**
- 3D coordinates are mathematically transformed into 2D screen coordinates using:
  - Viewing transformations
  - Projection transformations

2.4 Identification of Visible Parts

- Determines **which surfaces or edges are visible** to the viewer
- Techniques include:
  - Back-face culling
  - Z-buffer algorithm
  - Painter’s algorithm
- Prevents drawing hidden surfaces

2.5 Lighting Effects and Surface Characteristics

- Adds realism using:
  - Light sources
  - Material properties
  - Surface normals
- Common lighting models:
  - Ambient lighting
  - Diffuse reflection

- Specular reflection

---

3. Coordinate Reference for Viewing a 3D Scene

To obtain a specific view of a 3D scene, **multiple coordinate systems** are used:

Coordinate System	Purpose
Modeling Coordinates (MC)	Object's local coordinate system
World Coordinates (WC)	Global scene reference
Viewing Coordinates (VC)	Camera-based coordinates
Projection Coordinates (PC)	After projection
Normalized Coordinates (NC)	Clipping and normalization
Device Coordinates (DC)	Final screen coordinates

---

4. 3D Viewing – Camera Analogy

The entire viewing process is analogous to **taking a photograph with a camera**.

Step-by-Step Analogy

1. Set up the camera (Viewing Transformation)

- Decide:
  - Camera position
  - Camera orientation
- Equivalent to defining the **viewing coordinate system**

---

2. Arrange the scene (Modeling Transformation)

- Position objects correctly:
  - Translation
  - Rotation
  - Scaling
- Converts **model coordinates → world coordinates**

---

3. Choose a camera lens (Projection Transformation)

- Determines:
  - Perspective projection

    - Parallel (orthographic) projection
- Defines how depth is represented

---

4. Decide final image size (Viewport Transformation)

- Maps the image to:
    - A specific region of the screen
- Converts normalized coordinates to pixel coordinates

After these steps, the scene is ready to be **rendered or drawn**.

---

5. Photographing a Scene (Camera Position & Orientation)

- The appearance of a scene depends on:
    - Camera position
    - Viewing direction
    - Orientation of the camera
- Changing the camera position changes:
    - Visible surfaces
    - Depth perception
    - Relative object sizes

---

6. Projections

6.1 Parallel Projection

- Projection lines are **parallel**
- No perspective distortion
- Object size remains constant regardless of depth
- Used in:
    - Engineering drawings
    - CAD systems

Multiple parallel views show the object from different directions without distortion.

6.2 Perspective Projection (Implied)

- Projection lines converge at a point
- Objects farther away appear smaller

- Used in:
  - Games
  - Simulations
  - Realistic rendering

---

7. Wire-Frame Representation & Depth Cueing

Wire-Frame Representation

- Object is drawn using only:
  - Edges
  - Vertices
- No surfaces are filled

Depth Cueing

- Simulates depth by:
  - Reducing brightness for distant objects
  - Increasing brightness for closer objects
- Helps visualize 3D depth in wireframe models

---

8. Other Important 3D Viewing Concepts

8.1 Visible Line and Surface Detection

- Determines which parts should be drawn
- Avoids drawing hidden edges/surfaces

8.2 Surface Rendering

- Converts geometric surfaces into shaded images
- Includes:
  - Flat shading
  - Gouraud shading
  - Phong shading

8.3 Exploded and Cutaway Views

- **Exploded view**: Parts are separated to show structure
- **Cutaway view**: Portions removed to show interior

8.4 3D and Stereoscopic Viewing

- Uses two slightly different views
- Mimics human binocular vision
- Used in:
  - VR
  - 3D movies
  - Simulators

---

9. 3D Viewing Pipeline (Core Concept)

The **3D viewing pipeline** transforms coordinates step by step:

Pipeline Flow

```
nginx

MC → WC → VC → PC → NC → DC
```

Explanation of Each Stage

1. Modeling Transformation

- MC → WC
- Positions individual objects into the world

---

2. Viewing Transformation

- WC → VC
- Moves the world relative to the camera

---

3. Projection Transformation

- VC → PC
- Applies perspective or parallel projection

---

4. Normalization & Clipping

- PC → NC
- Removes objects outside the view volume
- Coordinates mapped to a standard range

5. Viewport Transformation

- NC → DC
- Converts to actual screen pixels

---

10. View-Plane Normal Vector (N)

Definition

- The **view-plane normal vector (N)** defines the direction in which the camera is looking.

Characteristics

- Perpendicular to the view plane
- Defines the **z-view axis**
- Uses a **right-handed coordinate system**

Orientation of the View Plane

- The view plane can be positioned:
  - In front of the camera
  - At the camera
  - Behind the camera
- Its orientation affects:
  - Visibility
  - Projection results

Specifying the View-Plane Normal

- Defined as the direction from:
  - Reference point **`Pref`**
  - To viewing origin **`P0`**
- Direction vector:

```ini
N = P0 – Pref
```

---

11. View-Up Vector (V)

Purpose

- Defines the **vertical orientation** of the camera

Adjustment

- Must be perpendicular to the view-plane normal
- Ensures the image is not rotated incorrectly

---

12. uvn Reference Frame (Viewing Coordinate System)

A **right-handed viewing coordinate system** defined by:

Vector	Meaning
n	View-plane normal (view direction)
u	Right direction (horizontal axis)
v	Up direction (vertical axis)

Properties

- All vectors are:
  - Unit vectors
  - Mutually perpendicular
- Forms the basis for viewing transformations

---

13. Summary

- 3D viewing converts a mathematical scene into a visible image
- Uses multiple coordinate systems and transformations
- Closely follows the camera analogy
- The viewing pipeline ensures:
  - Correct positioning
  - Proper projection
  - Accurate rendering
- View-plane normal, view-up vector, and uvn frame define camera orientation precisely

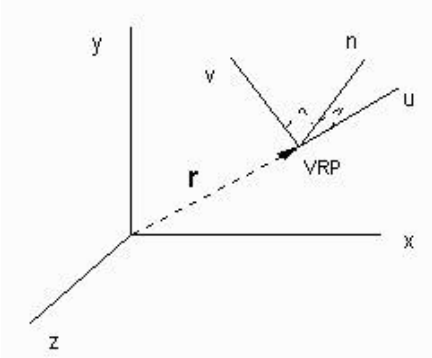
---

If you want, I can also:

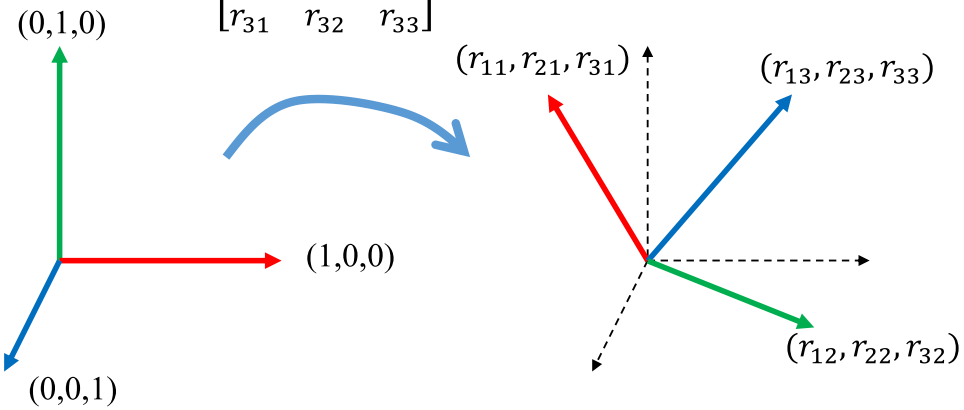
- Convert this into **exam-ready notes**
- Add **numerical examples**
- Provide **diagrams with formulas**

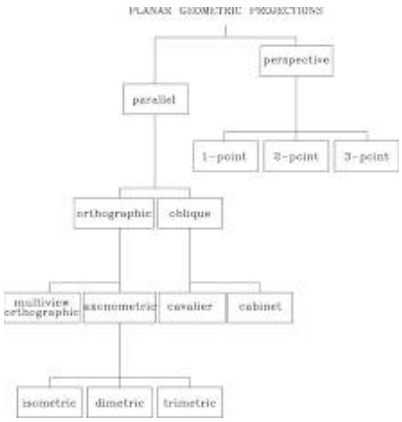
- Explain **perspective vs parallel projection mathematically**

Just tell me.



$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$





Below is a **complete, exam-ready, point-by-point explanation** of **Modeling Transformation, World → Viewing Coordinate Transformation, and Projection Transformations**, strictly following the order and content of your slides **without missing any concept**.

---

1. Modeling Transformation

Purpose

The **modeling transformation** is used to **position, orient, and size individual objects** in a 3D scene.

Each object is first defined in its **own local coordinate system**, called **model coordinates**.

Operations Involved

Modeling transformation is achieved using one or more of the following:

1. Rotation

- Changes the orientation of the object
- Used to align the object correctly in the scene

2. Scaling

- Changes the size of the object
- Can be:
  - Uniform (same scale factor in all directions)
  - Non-uniform (different scale factors)

Example:

```scss
glScalef(1.0, 2.0, 1.0);
```

This doubles the object size along the y-axis.

---

3. Translation

- Moves the object from one position to another
- Does not change shape or orientation

Important Observation

Instead of:

- Moving the camera backward to view an object

You can equivalently:

- Move the object away from the camera using modeling transformation

This is a fundamental concept in computer graphics:
moving the world is equivalent to moving the camera in the opposite direction.

---

2. Transformation from World to Viewing Coordinates

Why is This Needed?

Objects placed in **world coordinates** must be transformed into a coordinate system **relative to the viewer (camera)**.

This coordinate system is called the **viewing coordinate system**.

Viewing Coordinate System

- Origin is at the **camera position**
- Axes:
  - x_view → right
  - y_view → up
  - z_view → viewing direction

Panning and Viewing Direction

- **Panning** moves the viewing position sideways while keeping the viewing direction fixed
- Viewing the same object from different directions changes its projected appearance

---

3. How World → Viewing Transformation Is Achieved

This transformation is accomplished using **coordinate system transformations**, consisting of **two major steps**.

Step 1: Translation

Objective

Move the **viewing coordinate origin** to coincide with the **world origin**.

Given

Viewing origin is at:

```
ini

P0 = (x0, y0, z0)
```

Translation Matrix

To move the viewing origin to the world origin:

$$T = \begin{bmatrix} 1 & 0 & 0 & -x_0 \\ 0 & 1 & 0 & -y_0 \\ 0 & 0 & 1 & -z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This matrix:

- Translates the entire scene by **−P0**
- Effectively places the camera at the origin

Step 2: Rotation

Objective

Align the **viewing axes** with the **world axes**

---

uvn Reference Frame

A right-handed viewing coordinate system defined by three **unit vectors**:

Vector	Meaning
u	Right direction
v	Up direction
n	View-plane normal (view direction)

Rotation Matrix Using uvn Vectors

$$R = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ n_x & n_y & n_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Each row contains the components of:

- u vector
- v vector
- n vector

Final World → Viewing Transformation Matrix

The complete transformation is the **product of rotation and translation**:

$$M_{wc \to vc} = R \cdot T$$

Expanded form:

$$\begin{bmatrix} u_x & u_y & u_z & -u \cdot P_0 \\ v_x & v_y & v_z & -v \cdot P_0 \\ n_x & n_y & n_z & -n \cdot P_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Interpretation of Translation Terms

- Translation components are:
  - Negative projections of **P0** onto u, v, and n
- They represent the **camera position expressed in viewing coordinates**

---

4. Projection Transformation

Camera Lens Analogy

Projection transformation is equivalent to **choosing a camera lens**.

It determines:

- Field of view
- Viewing volume
- What part of the scene is visible

Lens Types Analogy

- **Wide-angle lens**
  - Larger field of view
  - More objects visible
  - More distortion
- **Telephoto lens**
  - Narrow field of view
  - Objects appear closer
- **Normal lens**
  - Balanced view

---

5. Geometric Projections

Definition

A **projection** is:

> An operation that transforms points in N-dimensional space into a space of lower dimension.

Types of Geometric Projections

1. Planar Projections

- Project onto a **plane**
- Most common in computer graphics

2. Non-Planar Projections

- Project onto curved surfaces
- Used in:
  - Map projections
  - Scientific visualization

Properties of Planar Projections

- Projectors are lines that:
  - Either **converge at a point**
  - Or remain **parallel**
- Lines are preserved
- Angles are **not always preserved**

---

6. Taxonomy of Planar Geometric Projections

Two Major Categories

A. Parallel Projections

- Projectors are parallel
- No perspective distortion

Types

1. **Orthographic**
   - Multiview (front, top, side)
   - Used in engineering drawings
2. **Axonometric**

- Isometric
- Dimetric
- Trimetric

3. **Oblique**
   - One face parallel to projection plane
   - Depth projected at an angle

B. Perspective Projections

- Projectors converge at a **center of projection**
- Realistic appearance

---

7. Perspective Projection

Key Characteristics

- Objects farther away appear smaller (diminution)
- Parallel lines not parallel to projection plane meet at **vanishing points**
- Depth perception is realistic

Vanishing Points

- Parallel edges converge to a point in the projection
- Used in hand-drawn perspective sketches

---

8. Types of Perspective Projections

One-Point Perspective

- One principal face parallel to projection plane
- One vanishing point
- Used for corridors, roads

Two-Point Perspective

- One principal direction parallel
- Two vanishing points
- Used for corner views of buildings

Three-Point Perspective

- No principal face parallel
- Three vanishing points
- Used for dramatic viewpoints (looking up/down)

---

9. Perspective vs Parallel Projection

Aspect	Parallel	Perspective
Realism	Low	High
Size change with depth	No	Yes
Vanishing points	None	One or more
Ease of construction	Easy	Hard by hand
Used in CAD	Yes	Rare

Computer graphics uses a **single pipeline** for both, even though classical drawing treated them differently.

---

10. Advantages and Disadvantages of Perspective Projection

Advantages

- Realistic appearance
- Natural depth perception
- Matches human vision

Disadvantages

- Equal distances do not remain equal (non-uniform foreshortening)
- Angles preserved only in planes parallel to projection plane
- Harder to construct manually

---

11. Summary

- Modeling transformation positions objects
- World → Viewing transformation defines camera position and orientation
- uvn frame establishes viewing axes
- Projection transformation defines viewing volume

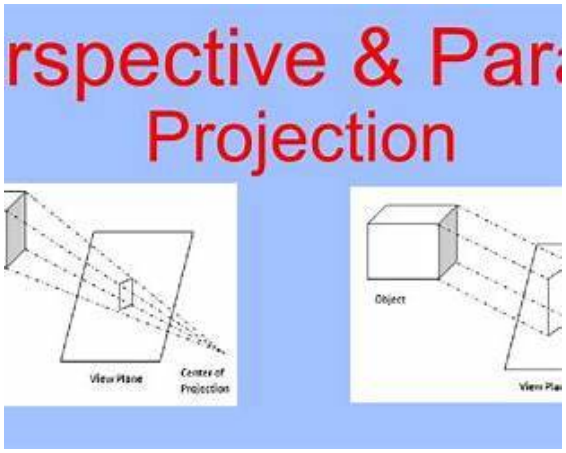
- Planar projections include parallel and perspective
- Perspective projection introduces realism using vanishing points

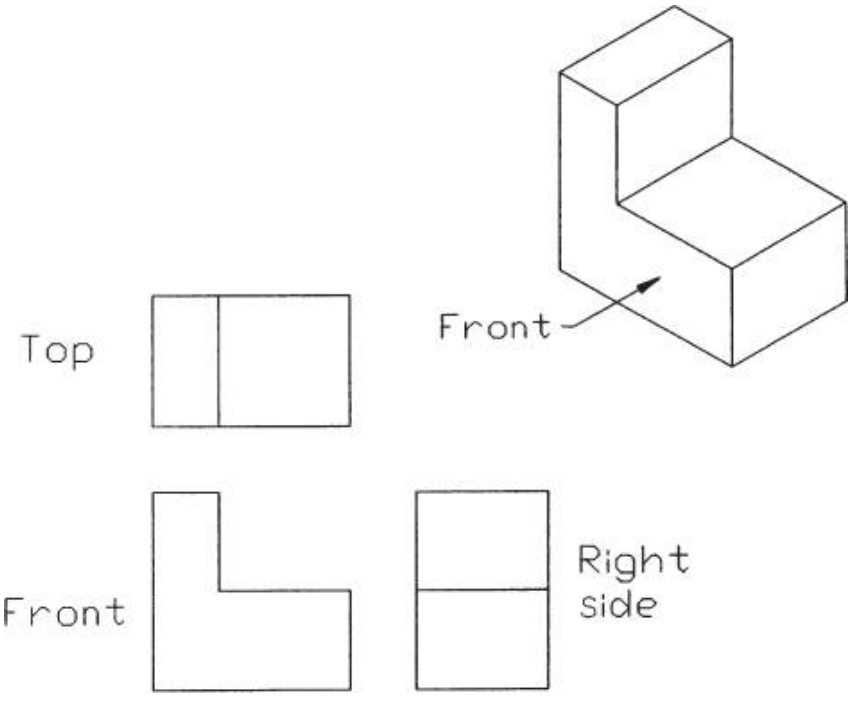
---

If you want, I can next:

- Convert this into **short exam answers**
- Add **numerical problems**
- Provide **derivations with diagrams**
- Create a **comparison table PDF**

Just tell me.





https://www.qpractice.com/wp-content/uploads/PSA-Axonometric_Projections_Recolored.png

Below is a **complete, structured, and exam-oriented explanation** of **Parallel Projection, Orthographic / Axonometric / Oblique Projections, Projection Transformation, Clipping, View**

Volumes, and OpenGL Projection Functions, following **exactly the sequence and intent of your slides**, without omitting any point.

---

1. Parallel Projection

Definition

In **parallel projection**, all projection lines (projectors) are **parallel to each other**.

- There is **no center of projection**
- Object size does **not change with distance**
- Depth does not affect scale

Key Property

- **Proportions are preserved**
- Used where **accurate measurements** are more important than realism

---

Two Ways to Obtain Parallel Projection

1. **Projectors perpendicular to the view plane**
   → Orthographic projection
2. **Projectors inclined at an angle to the view plane**
   → Oblique projection

---

2. Orthographic Projection

Definition

In **orthographic projection**, the projectors are:

- **Perpendicular (orthogonal)** to the projection plane

Characteristics

- No perspective distortion
- Parallel edges remain parallel
- True shape and size preserved on planes parallel to projection plane

---

Applications

- Engineering drawings
- Architecture
- CAD systems
- Blueprints and manuals

---

3. Multiview Orthographic Projection

Concept

- Projection plane is **parallel to a principal face** of the object
- Multiple orthographic views are generated

Common Views

- **Front view**
- **Top view**
- **Side view**

These are called **multiview orthographic projections**.

Important Note

- **Isometric projection is NOT a multiview orthographic view**
- In practice (CAD & architecture):
  - Three multiviews (front, top, side)
  - Plus one **isometric view** for visualization

---

4. Advantages and Disadvantages of Multiview Orthographic Projection

Advantages

- Preserves:
  - Distances
  - Angles
  - Shapes
- Can be used for **exact measurements**
- Ideal for:

- Building plans
- Assembly manuals

Disadvantages

- Does not show realistic appearance
- Many surfaces are hidden in a single view
- Requires multiple views to understand the object fully
- Often supplemented with an **isometric view**

---

5. Axonometric Projections

Definition

Axonometric projections:

- Allow the **projection plane to move relative to the object**
- Show **more than one face** of the object simultaneously

Classification Criterion

Based on how many angles of a cube corner are equal after projection:

Type	Equal Angles
Trimetric	None
Dimetric	Two
Isometric	Three

Orientation Angles

The three axes are projected at angles:

- q_1
- q_2
- q_3

---

6. Types of Axonometric Projections

1. Isometric Projection

- All three axes equally inclined
- Equal foreshortening along all axes
- Most widely used axonometric projection

---

2. Dimetric Projection

- Two axes have equal foreshortening
- One axis differs
- Less common than isometric

---

3. Trimetric Projection

- All three axes have different foreshortening
- Most flexible
- Least commonly used

---

7. Advantages and Disadvantages of Axonometric Projections

Advantages

- Lines are preserved
- Scaling factors can be computed
- Three principal faces visible
- Useful for CAD visualization

Disadvantages

- Angles are not preserved
- Circles appear as ellipses
- Optical illusions may occur
    - Parallel lines may appear to diverge
- Not realistic:
    - Far and near objects appear the same size

---

8. Oblique Projection

Definition

In **oblique projection**:

- Projectors are at an **arbitrary angle** to the projection plane
- One face is parallel to projection plane

Key Property

- Front face appears in true shape
- Depth is projected at an angle

Classical Oblique Projections

- Cavalier
- Cabinet

---

9. Advantages and Disadvantages of Oblique Projection

Advantages

- Angles on the front face are preserved
- Depth visible while maintaining front-face accuracy
- Useful in:
  - Architecture
  - Elevation drawings
  - Plan oblique views

Disadvantages

- Not physically realistic
- Cannot be produced with a simple camera
- Requires special lenses or techniques

---

10. Projection Transformation (Summary)

Two Basic Projection Types

1. **Parallel Projection**
   - Proportions preserved
   - Includes orthographic and oblique

2. **Perspective Projection**
    - Matches human vision
    - Objects farther away appear smaller

---

11. Orthographic Projection in Detail

Types of Orthographic Views

- **Elevation views**
    - Front
    - Side
    - Rear
- **Plan view**
    - Top view

Use Case

Used where:

- Exact dimensions are required
- Visual realism is not important

---

12. Orthographic View Volume

Characteristics

- Rectangular clipping window
- Finite view volume
- Defined by:
    - Left, Right
    - Top, Bottom
    - Near, Far planes

Near–Far Clipping Planes

- Limit the visible depth range
- Objects outside this volume are clipped

---

13. Orthogonal Projection Coordinates

When projection direction is **parallel to z_view axis**:

$$x_p = x$$

$$y_p = y$$

These equations are trivial because depth does not affect x and y.

---

14. Clipping Window and Orthographic Projection

Clipping Window

- Usually rectangular
- Defines visible region in projection plane

Clipping Volume

- Extended into 3D using near and far planes
- Results in a **rectangular box-shaped view volume**

---

15. Viewport Transformation and 3D Screen Coordinates

Purpose

Maps normalized view volume coordinates to **device (screen) coordinates**

Similarity with 2D

- Same concept as 2D viewport mapping
- Extended to include depth (z)

Result

- Converts normalized coordinates into:
  - Pixel coordinates
  - Screen depth values

---

16. Projection in OpenGL

Two Projection Types Supported

1. **Perspective projection**
2. **Orthographic projection**

---

17. Perspective Projection in OpenGL

glFrustum()

Defines a perspective viewing volume manually:

```
css

glFrustum(left, right, bottom, top, near, far);
```

- Viewing volume is a **truncated pyramid**
- near > 0

---

gluPerspective()

Defines perspective projection using intuitive parameters:

```
scss

gluPerspective(fovy, aspect, near, far);
```

Where:

- fovy → vertical field of view
- aspect → width / height
- near, far → clipping planes

---

18. OpenGL Projection Setup Example

```
scss

glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glFrustum(-1.0, 1.0, -1.0, 1.0, 1.5, 20.0);
```

This:

- Switches to projection matrix
- Resets it
- Defines a perspective frustum

19. Summary

- Parallel projection preserves proportions
- Orthographic projection preserves measurements
- Axonometric shows multiple faces
- Oblique emphasizes a selected face
- Perspective projection adds realism
- Clipping defines visible volume
- Viewport maps to screen
- OpenGL supports both projection types

If you want next:

- **Numerical problems**
- **Derivation questions**
- **Comparison tables**
- **Exam-ready short answers**
- **MCQs with explanations**

Just tell me.

Vertex (in homogenous coordinates): $\boldsymbol{V} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$

Translation: $\boldsymbol{T}(d) = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$ where $d = \begin{bmatrix} d_x \\ d_y \\ d_z \\ 1 \end{bmatrix}$ is the tranlational vector

The Inverse $\boldsymbol{T^{-1}}(d) = \boldsymbol{T}(-d)$

Scaling: $\boldsymbol{S}(\alpha_x, \alpha_y, \alpha_z) = \begin{bmatrix} \alpha_x & 0 & 0 & 0 \\ 0 & \alpha_y & 0 & 0 \\ 0 & 0 & \alpha_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ where $\alpha_x, \alpha_y, \alpha_z$ are the scaling factor

Uniform Scaling: $\alpha_x = \alpha_y = \alpha_z$

The inverse $\boldsymbol{S^{-1}}(\alpha_x, \alpha_y, \alpha_z) = \boldsymbol{S}\left(\frac{1}{\alpha_x}, \frac{\mathbf{1}}{\alpha_y}, \frac{1}{\alpha_z}\right)$

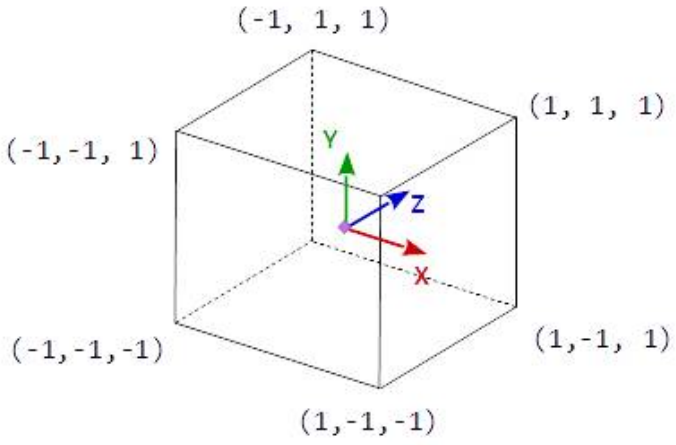
Rotation: $\boldsymbol{R_z}(\theta) = \begin{bmatrix} cos\theta & -sin\theta & 0 & 0 \\ sin\theta & cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \boldsymbol{R_y}(\theta) = \begin{bmatrix} cos\theta & 0 & sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -sin\theta & 0 & cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \boldsymbol{R_x}(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos\theta & -sin\theta & 0 \\ 0 & sin\theta & cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

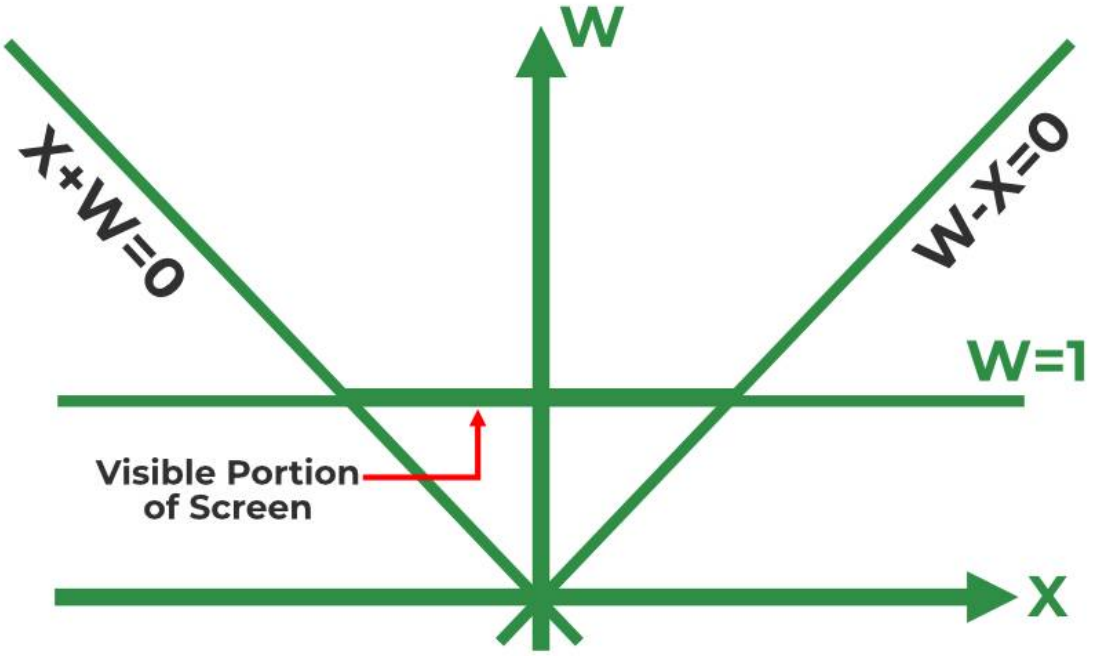
The inverse $\boldsymbol{R_z^{-1}}(\theta) = \boldsymbol{R_z}(-\theta)$

Shearing: $\boldsymbol{H_x}(\theta) = \begin{bmatrix} 1 & cot\theta & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ where θ is the shearing angle

The inverse $\boldsymbol{H_x^{-1}}(\theta) = \boldsymbol{H_x}(-\theta)$

Normalized Device Coordinates (NDC)





Below is a **complete, structured, exam-oriented explanation** of **OpenGL 3D Viewing Functions, Orthographic Projection, Viewport Transformation, Drawing Pipeline, and 3D Clipping**, explained **step by step**, **in order**, and **without missing any point** from your slides.

---

1. OpenGL Three-Dimensional Viewing Functions (Table 10-1)

OpenGL provides a set of **standard functions** to define the **viewing, projection, and clipping behavior** of a 3D scene.

1.1 `gluLookAt`

Purpose

Specifies the **three-dimensional viewing parameters** (camera setup).

What it defines

- Camera (eye) position
- Look-at point (reference point)
- View-up direction

Conceptually

It internally constructs the **viewing transformation matrix** using the **uvn reference frame**.

1.2 `glOrtho`

Purpose

Defines parameters for an **orthographic (parallel) projection**.

What it specifies

- Clipping window (left, right, bottom, top)
- Near and far clipping planes

Effect

- Objects are projected **without perspective distortion**
- Object size does not change with depth

1.3 `gluPerspective`

Purpose

Defines a **symmetric perspective projection**.

What it specifies

- Field-of-view angle
- Aspect ratio
- Near and far clipping planes

Effect

- Objects farther away appear smaller
- Mimics human vision

1.4 `glFrustum`

Purpose

Defines a **general perspective projection** (symmetric or oblique).

What it specifies

- Left, right, bottom, top of the near plane
- Near and far clipping planes

Effect

- Viewing volume is a **truncated pyramid (frustum)**

1.5 `glClipPlane`

Purpose

Defines an **optional user-defined clipping plane**.

Used for

- Cutting objects
- Cross-sectional views
- Advanced visualization

---

2. Orthographic Projection in OpenGL

Function Syntax

```
css

glOrtho(left, right, bottom, top, near, far)
```

Characteristics

- Projection lines are **parallel**
- Measurements are preserved
- No foreshortening
- Common in CAD and architectural drawings

Interpretation of Parameters

- **`left`**, **`right`** → horizontal limits
- **`bottom`**, **`top`** → vertical limits
- **`near`**, **`far`** → depth limits

---

3. Viewport Transformation

Purpose

The **viewport transformation** maps the projected scene onto a **specific region of the screen**.

Key Concept

- **Projection transformation** decides *how* the scene is projected

- **Viewport transformation** decides *where* on the screen it appears

---

OpenGL Function

```
css

glViewport(x, y, width, height)
```

Parameters

- `(x, y)` → lower-left corner of viewport
- `width`, `height` → size of viewport rectangle

Importance

- Enables multiple views in one window
- Controls screen-space placement

---

4. Drawing the Scene (Rendering Pipeline)

Once all transformations are specified, OpenGL proceeds to **draw the scene**.

Step-by-Step Rendering Process

1. Each vertex is transformed by:
    - Modeling transformation
    - Viewing transformation
2. Then transformed by:
    - Projection transformation
3. **Clipping**:
    - Vertices outside the viewing volume are removed
4. **Perspective division**:
    - Each vertex is divided by its homogeneous coordinate `w`
5. **Viewport mapping**:
    - Final screen coordinates are obtained

---

5. 3D Clipping – Core Idea

Main Idea

3D clipping is an **extension of 2D clipping concepts into 3D space**.

Assumptions

- View volume is normalized
- Clipping is performed against:
  - Unit cube [0,1] or
  - Symmetric cube [−1,1]

Why Clipping Is Needed

- To remove:
  - Invisible objects
  - Objects outside the camera's view
- Improves efficiency and correctness

---

6. Homogeneous Coordinates and Clipping

Homogeneous Representation

A 3D point:

```
css

(x, y, z) → (x, y, z, h)
```

Important Issue

- After transformations, $h \neq 1$
- Direct clipping after dividing by h can:
  - Lose information
  - Cause incorrect clipping

Solution

Perform clipping in homogeneous coordinates, *before* division by h.

---

7. Normalized Symmetric Clipping Volume

A point is **inside** the view volume if:

$$-1 \leq \frac{x_h}{h} \leq 1$$

$$-1 \leq \frac{y_h}{h} \leq 1$$

$$-1 \leq \frac{z_h}{h} \leq 1$$

If `h ≠ 0`, this becomes:

For `h > 0`

$$-h \leq x_h \leq h$$

$$-h \leq y_h \leq h$$

$$-h \leq z_h \leq h$$

For `h < 0`

$$h \leq x_h \leq -h$$

$$h \leq y_h \leq -h$$

$$h \leq z_h \leq -h$$

Meaning

Clipping conditions depend on the **sign of h**, which is why clipping must be done **before perspective division**.

---

8. 3D Region Codes (Cohen–Sutherland Extension)

To efficiently clip objects, **region codes** are used.

6-Bit Region Code (3D)

Bit	Condition	Plane
1	h + x < 0	Left
2	h − x < 0	Right
3	h + y < 0	Bottom
4	h − y < 0	Top
5	h + z < 0	Near
6	h − z < 0	Far

Interpretation

- `000000` → point is inside
- Any other value → point is outside or partially outside

For `h < 0`

- Conditions are reversed
- Same logic applies with sign changes

9. 3D Clipping Algorithm

Point Clipping

- Discard points with region code ≠ `000000`

Line Clipping

1. **Trivial accept**
   - Logical OR of codes = 0
2. **Trivial reject**
   - Logical AND of codes ≠ 0
3. Otherwise:
   - Use **Cyrus–Beck** or **Liang–Barsky** algorithms

Polygon and Curve Clipping

- Extend line clipping concepts
- Clip edges and reconstruct polygons

10. 3D Clipping in OpenGL

Defining a Clipping Plane

```scss
glClipPlane(id, planeParameters);
glEnable(id);
```

Disabling

```scss
glDisable(id);
```

Plane Parameters

- Defined by coefficients **A, B, C, D**
- Plane equation:

$$Ax + By + Cz + D = 0$$

Clipping Rule

- If:

$$Ax + By + Cz + D < 0$$

The point is **discarded**

Reason

The point lies **behind the clipping plane**, relative to the plane's normal.

11. Summary

- OpenGL provides standard viewing and projection functions
- Orthographic projection preserves measurements
- Viewport transformation maps to screen space
- Rendering pipeline applies transformations → clipping → mapping
- 3D clipping is performed in homogeneous space
- Region codes extend Cohen–Sutherland to 3D
- OpenGL supports custom clipping planes

If you want, I can now:

- Convert this into **exam-ready answers**
- Provide **stepwise derivations**
- Create **comparison tables**
- Generate **MCQs with solutions**
- Explain **numerical problems**

Just tell me what you need next.